Machine Learning — Unit 4 Exam Answers

Unit 4: Reinforcement Learning (Syllabus Unit VI)

SPPU B.E. (AI & DS) 2019 Pattern, Course 417521 · Exam: 70 marks, 2.5 hours · Paper unit = question pair (Q1/Q2 → Unit 1, Q3/Q4 → Unit 2, Q5/Q6 → Unit 3, Q7/Q8 → Unit 4).

Each answer is written for **(X + 2) marks**, where X = highest marks ever asked. Sources cite your PDFs by file name and PDF page number.

1. Unit Snapshot

No.	Question	Max Marks	Years Asked	Importance
U4-1	What is Reinforcement Learning? Explain its need/importance, types and real-time applications.	9	Nov-Dec 2023, Nov-Dec 2024, Nov-Dec 2025	★★★
U4-2	Compare supervised, unsupervised and reinforcement learning with examples.	9	Nov-Dec 2023, Nov-Dec 2024, Nov-Dec 2025	★★★
U4-3	Explain the Q-Learning algorithm: important terms, working, and an example.	9	Nov-Dec 2023, Nov-Dec 2025	★★★
U4-4	Explain the Markov property and Markov Decision Process (MDP). How does the Markov property relate to RL and why is it important?	9	Nov-Dec 2024, Nov-Dec 2025	★★★
U4-5	Explain the terms: Markov Property, Markov Chain, Markov Reward Process, Bellman Equation, Q-table, Q-function.	8	Nov-Dec 2023, Nov-Dec 2024	★★★

2. Must-Do First (★★★)

1. **U4-1** — What is Reinforcement Learning? Explain its need/importance, types and real-time applications. (9 M)
2. **U4-2** — Compare supervised, unsupervised and reinforcement learning with examples. (9 M)
3. **U4-3** — Explain the Q-Learning algorithm: important terms, working, and an example. (9 M)
4. **U4-4** — Explain the Markov property and Markov Decision Process (MDP). How does the Markov property relate to RL and why is it important? (9 M)
5. **U4-5** — Explain the terms: Markov Property, Markov Chain, Markov Reward Process, Bellman Equation, Q-table, Q-function. (8 M)

3. Full Answers

U4-1. What is Reinforcement Learning? Explain its need/importance, types and real-time applications.

Importance: ★★★ | **Max asked:** 9 M → **write for 11 M** | **Asked in:** Nov-Dec 2023, Nov-Dec 2024, Nov-Dec 2025

Source: ML Unit 3-6.pdf, pp. 87–94 (Sec. 6.1 What is RL, need, elements, real-time applications; Sec. 6.2–6.3 types, positive vs negative reinforcement; book pp. 6-2 to 6-9).

Asked as: *What is Reinforcement Learning? Explain the real time applications of reinforcement learning.* (Nov-Dec 2023 Q8(a), 8 M) | *Explain Reinforcement Learning need and its types in detail?* (Nov-Dec 2024 Q7(b), 9 M) | *Define reinforcement learning. Why is reinforcement learning important in the field of machine learning?* (Nov-Dec 2025 Q7(a), 9 M)

Definition

Reinforcement Learning (RL) is a type of machine learning in which an **agent** learns to take **actions** in an **environment** by **trial and error**, receiving **rewards or penalties**, so as to **maximise the cumulative (long-term) reward**. No labelled data is given – only a **reward signal**.



Fig. Agent–environment interaction loop in RL

- **Agent:** learner/decision maker (follows policy π).
- **Environment:** world the agent acts in.
- **Action a_t:** move chosen by the agent. **State s_{t+1}:** new situation returned by environment.
- **Reward r_{t+1}:** numerical feedback (+ve = good, −ve = bad).

1. Elements of RL

- **Agent, Environment, State (s), Action (a), Reward (r)**.
- **Policy π(a|s):** strategy mapping states to actions.
- **Value function V(s):** expected long-term return from a state.
- **Q-value Q(s,a):** expected return of taking action a in state s.
- **Model (optional):** agent's prediction of environment's next state/reward.
- **Discount factor γ (0–1):** importance of future rewards.

$$\text{Return } G_t = r_{t+1} + \gamma\, r_{t+2} + \gamma^2\, r_{t+3} + \dots$$

2. Need / Importance of RL

1. Learns **without labelled data** – only feedback from interaction.
2. Solves **sequential decision-making** problems where actions affect future states.
3. Handles **delayed rewards** (e.g., winning a game only at the end).
4. **Adapts** to dynamic, uncertain environments; continuous learning.
5. Balances **exploration vs exploitation** – discovers new strategies.
6. Achieves **super-human performance** (AlphaGo, Atari games) and real-world control.

3. Types of Reinforcement Learning

1. **Positive reinforcement:** a reward is given for good behaviour → increases frequency of that behaviour (maximises performance; too much may overload states).
2. **Negative reinforcement:** behaviour is strengthened by **removing/avoiding a negative condition** (e.g., penalty) → ensures minimum standard of performance.

- *By approach:*
  - **Model-based vs model-free** (Q-learning, SARSA are model-free).
  - **Value-based** (Q-learning, DQN), **policy-based** (REINFORCE), **actor–critic**.
  - **On-policy** (SARSA) vs **off-policy** (Q-learning).

4. Real-Time Applications

Area	Application
Games	**AlphaGo**, chess, Atari – learn winning strategies
Robotics	Robot walking, grasping objects, industrial automation
Self-driving cars	Lane keeping, parking, trajectory planning
Finance	**Stock trading**, portfolio management
Healthcare	Dynamic treatment plans, drug dosage
Recommendation	News/video recommendations (YouTube, Netflix)
Resource management	Data-centre cooling (Google), traffic-light control
NLP / Chatbots	Dialogue systems, **RLHF** for large language models

Example

A dog learns tricks: it **sits** (action) after the command (state); gets a **biscuit** (reward); it learns to repeat the action. Similarly a robot learns to walk by getting +1 for each step forward and −10 for falling.

Conclusion

RL lets an agent learn **optimal behaviour from rewards** through interaction; it is essential for sequential decision problems such as games, robotics and autonomous driving.

Memory trick – "A-E-S-A-R": Agent in **E**nvironment sees **S**tate, takes **A**ction, gets **R**eward. Types: **+ve** adds a reward, **–ve** removes a penalty.

U4-2. Compare supervised, unsupervised and reinforcement learning with examples.

Importance: ★★★ | **Max asked:** 9 M → **write for 11 M** | **Asked in:** Nov-Dec 2023, Nov-Dec 2024, Nov-Dec 2025

Source: ML Unit 3-6.pdf, p. 92 (Sec. 6.1.6 Difference between Supervised, Unsupervised and Reinforced Learning; book p. 6-7).

Asked as: *Explain following terms: i) Supervised Learning. ii) Unsupervised Learning. iii) Reinforcement Learning.* (Nov-Dec 2023 Q8(b), 9 M) | *Differentiate supervised and unsupervised learning with example.* (Nov-Dec 2024 Q7(a), 8 M) | *Compare and contrast supervised, unsupervised, and reinforcement learning. Provide examples of each type.* (Nov-Dec 2025 Q8(b), 8 M)

Definition

- **Supervised learning:** learns a mapping from **labelled data** (input → known output) to predict outputs for new inputs.
- **Unsupervised learning:** finds hidden **patterns/structure** in **unlabelled** data.
- **Reinforcement learning:** an **agent learns by interaction**, using **rewards** to find the best actions.

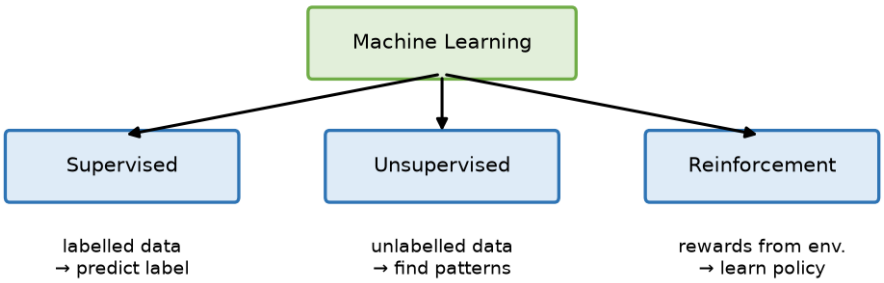


Fig. Three learning paradigms of machine learning

1. Supervised Learning

- Data: pairs (x, y) – a "teacher" provides correct answers.
- Tasks: **classification** (discrete y) and **regression** (continuous y).
- Algorithms: linear/logistic regression, KNN, SVM, decision trees, neural networks.

- **Example:** email **spam detection**; house-price prediction.

2. Unsupervised Learning

- Data: only x (no labels).
- Tasks: **clustering, association rules, dimensionality reduction**.
- Algorithms: K-Means, DBSCAN, hierarchical, PCA, Apriori.
- **Example: customer segmentation**; market-basket analysis.

3. Reinforcement Learning

- Data: generated by **agent–environment interaction**; feedback = reward.
- Tasks: **sequential decision making / control**.
- Algorithms: Q-learning, SARSA, DQN, policy gradient.
- **Example: game playing (AlphaGo)**, self-driving car, robot navigation.

4. Comparison

Basis	Supervised	Unsupervised	Reinforcement
Input data	**Labelled** (x, y)	**Unlabelled** (x)	No fixed dataset; **states from environment**
Feedback	Direct – correct answer	**None**	**Reward / penalty** (delayed)
Goal	Predict output / label	Discover patterns, groups	**Maximise cumulative reward**
Learning style	Learn from teacher (examples)	Self-organisation	**Trial and error**
Tasks	Classification, regression	Clustering, association, dim. reduction	Control, decision making
Algorithms	LR, SVM, KNN, DT	K-Means, DBSCAN, PCA, Apriori	Q-learning, SARSA, DQN
Decision type	Independent, one-shot	–	**Sequential**, each action affects future
Evaluation	Accuracy, MSE	Silhouette, WCSS	Total reward
Example	Spam filter, price prediction	Customer segmentation	Chess/AlphaGo, robot walking

Conclusion

Supervised learning learns **from answers**, unsupervised learning **finds structure**, and reinforcement learning **learns from rewards** by acting in an environment.

Memory trick: Supervised = **S**tudent with teacher; **U**nsupervised = **U**nguided explorer; **R**einforcement = **R**eward-driven pet.

U4-3. Explain the Q-Learning algorithm: important terms, working, and an example.

Importance: ★★★ | **Max asked:** 9 M → **write for 11 M** | **Asked in:** Nov-Dec 2023, Nov-Dec 2025

Source: ML Unit 3-6.pdf, pp. 99–101 (Sec. 6.5 Q Learning; book pp. 6-14 to 6-16). The numerical update example is a standard illustration.

Asked as: *Explain Q-Learning algorithm with an example.* (Nov-Dec 2023 Q7(b), 9 M) | *Introduce Q-learning. What are the important terms used in Q-learning? How does Q-learning work?* (Nov-Dec 2025 Q8(a), 9 M)

Definition

Q-Learning is a **model-free, off-policy, value-based** reinforcement learning algorithm that learns the **optimal action-value function Q*(s, a)** – the expected cumulative reward of taking action a in state s and acting optimally afterwards – using the **Bellman equation** and a **Q-table**.

1. Important Terms

- **State (s), Action (a), Reward (r), Next state (s')**.

- **Q-value Q(s,a):** "quality" of action a in state s.
- **Q-table:** table with rows = states, columns = actions, entries = Q-values (initialised to 0).
- **Learning rate α (0–1):** how much new information overrides old.
- **Discount factor γ (0–1):** importance of future rewards.
- **Episode:** one run from start state to terminal state.
- **ε-greedy policy:** with probability ε **explore** (random action), else **exploit** (best Q action).
- **Temporal Difference (TD) error:** r + γ max Q(s′,a′) − Q(s,a).

2. Q-Learning Update Rule (Bellman)

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \right]$$

- **r + γ max Q(s′,a′)** = target (TD target); difference from old Q = TD error.

3. Algorithm / How it works

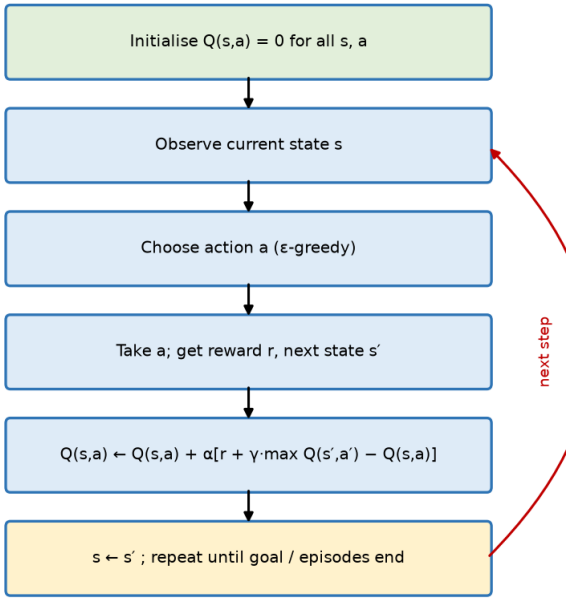


Fig. Q-Learning algorithm flowchart

1. **Initialise** Q(s, a) = 0 for all states and actions; set α, γ, ε.
2. For each **episode**: start at initial state s.
3. **Choose action a** using ε-greedy policy.
4. **Perform a**, observe reward **r** and next state **s′**.
5. **Update Q(s, a)** using the update rule.
6. Set **s ← s′**; repeat steps 3–6 until terminal state.
7. Repeat many episodes until Q-values **converge**; optimal policy **$\pi^*(s) = \text{argmax}_a\, Q(s, a)$**.

4. Example (one update)

Robot in a grid: state **S1**, action **Right** → reaches **S2**, reward **r = +10**. Given α = 0.5, γ = 0.9, Q(S1,Right) = 0, max Q(S2,·) = 5:

$$Q(\text{S1,Right}) = 0 + 0.5\,[10 + 0.9 \times 5 - 0] = 0.5 \times 14.5 = 7.25$$

State	Left	Right	Up	Down
S1	0	**7.25**	0	0
S2	0	5	0	0

Next time in S1, the agent prefers **Right** (highest Q).

5. Advantages / Disadvantages

- **+** Model-free (no environment model needed); off-policy (learns optimal policy while exploring); simple; guaranteed convergence with enough exploration.
- **–** Q-table becomes huge for large/continuous state spaces (→ **Deep Q-Network**); slow convergence; needs tuning of α, γ, ε.

6. Applications

- Game playing, robot navigation, traffic signal control, recommendation, trading bots.

Conclusion

Q-learning fills a **Q-table** by repeatedly applying the **Bellman update** while exploring, and finally acts greedily on the learned Q-values.

Memory trick: "**New Q = Old Q + α (Target – Old)**", Target = **r + γ·max next Q**. Off-policy = learns the **max**, even while acting ε-greedy.

U4-4. Explain the Markov property and Markov Decision Process (MDP). How does the Markov property relate to RL and why is it important?

Importance: ★★★ | **Max asked:** 9 M → **write for 11 M** | **Asked in:** Nov-Dec 2024, Nov-Dec 2025

Source: ML Unit 3-6.pdf, pp. 94–99 (Sec. 6.4 Markov Decision Process, Markov property, Bellman equation; book pp. 6-9 to 6-14).

Asked as: *How does the Markov property relate to Reinforcement Learning? Why is it important?* (Nov-Dec 2024 Q8(b), 9 M) | *Explain Markov's Decision Process (MDP). What is the Markov property in the context of MDP?* (Nov-Dec 2025 Q7(b), 8 M)

Definition

The **Markov property** states that the **future state depends only on the present state (and action), not on the past history**. A **Markov Decision Process (MDP)** is the mathematical framework of RL in which an agent makes decisions in states that satisfy the Markov property.

$$P(s_{t+1} \mid s_t, a_t) = P(s_{t+1} \mid s_1, a_1, \ldots, s_t, a_t)$$

- "**The future is independent of the past given the present.**"

1. MDP Components – tuple (S, A, P, R, γ)

- **S:** finite set of **states**.
- **A:** set of **actions**.
- **P(s' | s, a): transition probability** of reaching s' from s by action a.
- **R(s, a): reward** received after taking action a in state s.
- **γ: discount factor** ($0 \le \gamma \le 1$).
- **Policy π(a | s):** what the agent does in each state; goal = find **optimal policy π** *maximising expected return*

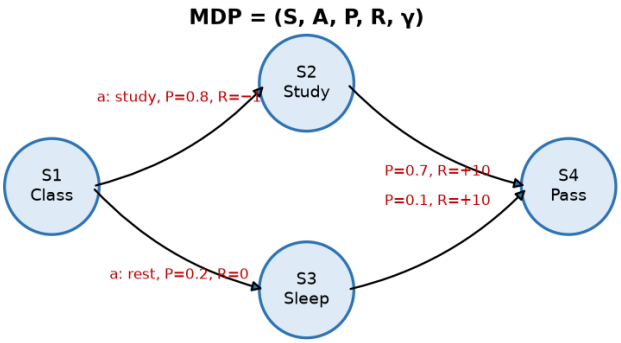


Fig. Simple MDP: states, actions, transition probabilities and rewards

- **Circles:** states (Class, Study, Sleep, Pass). **Arrows:** actions/transitions with probability P and reward R.
- Next state depends only on the current state and chosen action → Markov property.

2. Working of an MDP

1. Agent observes state $\mathbf{s_t}$.
2. Chooses action $\mathbf{a_t}$ by policy π.
3. Environment moves to $\mathbf{s_{t+1}}$ with probability $P(s_{t+1} \mid s_t, a_t)$ and gives reward $\mathbf{r_{t+1}}$.
4. Repeat; maximise expected return $\mathbf{G_t = \Sigma\, \gamma^k\, r_{t+k+1}}$.
5. Solve using **Bellman equations** (value iteration, policy iteration, Q-learning).

$$V^*(s) = \max_a \left[R(s,a) + \gamma \Sigma_{s'} P(s' \mid s,a)\, V^*(s') \right]$$

3. Relation of Markov Property to RL & Why it is Important

1. **Foundation of RL:** RL problems are modelled as MDPs; the Markov property makes the state a **sufficient statistic** of history.
2. **Simplifies decision making:** the agent needs only the **current state** to choose the best action – no need to store full history.
3. **Enables Bellman equations / recursion:** value of a state can be written in terms of values of next states → basis of **dynamic programming, Q-learning, TD learning**.
4. **Reduces computation & memory** – policies and value functions are functions of state only.
5. **Guarantees convergence** of algorithms like value iteration and Q-learning (under conditions).
6. If the property does not hold (partial observation), we use **POMDPs** or add history (e.g., stacking frames) to make the state Markov.

Example

Chess: the current board position contains all information needed for the next move – how the pieces arrived there does not matter → Markov. **Weather** with only today's state deciding tomorrow's is a Markov chain.

Conclusion

MDP = (S, A, P, R, γ) is the framework of RL, and the **Markov property** (future depends only on present) makes it solvable with Bellman equations.

Memory trick: MDP = "**SAP-R-γ**" (**S**tates, **A**ctions, **P**robabilities, **R**ewards, **γ**). Markov = "**Present is enough**".

U4-5. Explain the terms: Markov Property, Markov Chain, Markov Reward Process, Bellman Equation, Q-table, Q-function.

Importance: ★★★ | **Max asked:** 8 M → **write for 10 M** | **Asked in:** Nov-Dec 2023, Nov-Dec 2024

Source: ML Unit 3-6.pdf, pp. 95–101 (Markov property p. 95, Markov chain pp. 96–97, Markov reward process p. 98, Bellman equation pp. 98–99, Q-table / Q-function pp. 100–101; book pp. 6-10 to 6-16).

Asked as: *Explain following terms: i) Markov Property ii) Bellman Equation iii) Markov Reward Process iv) Markov Chain* (Nov-Dec 2023 Q7(a), 8 M) | *Explain following terms: i) Belman Equation ii) Markov Chain iii) Q table iv) Q function* (Nov-Dec 2024 Q8(a), 8 M)

Definition

These terms form the mathematical base of reinforcement learning: a **Markov chain** adds probabilities to states, a **Markov Reward Process** adds rewards, an **MDP** adds actions, and **Bellman equations / Q-functions** compute the value of states and actions.

1. Markov Property

- The next state depends **only on the current state**, not on past states.

$$P(s_{t+1} \mid s_t) = P(s_{t+1} \mid s_1, s_2, \ldots, s_t)$$

- Example: in Ludo/snake-ladder, next position depends only on present position and dice.

2. Markov Chain (Markov Process)

- A sequence of random states with the **Markov property**, defined by a tuple **(S, P)** – states and **transition probability matrix**.
- Each row of P sums to 1.

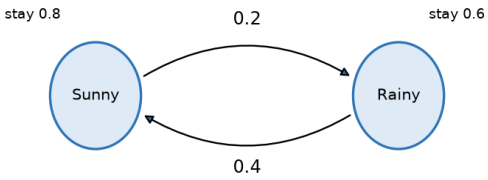


Fig. Two-state weather Markov chain

- **Example:** Sunny → Sunny 0.8, Sunny → Rainy 0.2; Rainy → Sunny 0.4, Rainy → Rainy 0.6.

3. Markov Reward Process (MRP)

- A Markov chain **with rewards**: tuple **(S, P, R, γ)**.
- **R(s)** = expected immediate reward in state s; **γ** = discount factor.
- Return: $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots$; state value $V(s) = E[G_t \mid s_t = s]$.
- **Example:** in the weather chain, reward +1 for Sunny, −1 for Rainy.

4. Bellman Equation

- Expresses the value of a state as **immediate reward + discounted value of next states** (recursive).

$$V(s) = R(s) + \gamma \, \Sigma_{s'} P(s' \mid s) \, V(s') \quad \text{(MRP)}$$

$$Q(s, a) = R(s, a) + \gamma \, \Sigma_{s'} P(s' \mid s, a) \, \max_{a'} Q(s', a') \quad \text{(Bellman optimality)}$$

- Used by dynamic programming and Q-learning to compute optimal values.

5. Q-Table

- A **lookup table** with **rows = states, columns = actions**; each cell stores **Q(s, a)**.
- Initialised to 0 and updated during Q-learning; the agent picks the action with the **highest value** in its row.

State	Up	Down	Left	Right
S1	0.2	0.0	0.1	**0.8**
S2	**0.6**	0.3	0.0	0.1

6. Q-Function (Action-Value Function)

- **Q^π(s, a)** = expected cumulative discounted reward of taking action a in state s and then following policy π:

$$Q^{\pi}(s, a) = E_{\pi} [\, G_t \mid s_t = s, a_t = a \,]$$

- Optimal Q-function *Q(s,a) gives the optimal policy:* $\pi(s) = \text{argmax}_a \, Q(s, a)^*$.
- Relation: $V(s) = \max_a Q(s, a)$ (for optimal values).

Conclusion

Markov chain (S, P) → MRP (S, P, R, γ) → MDP (S, A, P, R, γ); the Bellman equation computes values recursively and the Q-function/Q-table stores the value of each action in each state.

Memory trick: "**Chain → +Reward → +Action**" = MC → MRP → MDP. Q-table = "**cheat-sheet of best moves**".

4. Last-Minute Revision Sheet

Key Definitions

- **RL:** agent learns by trial & error to maximise cumulative reward.
- **Elements:** agent, environment, state, action, reward, policy, value, model.
- **Types:** positive & negative reinforcement; model-based/model-free; value/policy-based.
- **Markov property:** future depends only on present.
- **Markov chain:** (S, P); **MRP:** (S, P, R, γ); **MDP:** (S, A, P, R, γ).
- **Q-learning:** model-free, off-policy, Q-table + Bellman update, ε-greedy.

Formulas

Item	Formula
Return	$G_t = \Sigma_k \gamma^k r_{t+k+1}$
Markov property	$P(s_{t+1}\|s_t) = P(s_{t+1}\|s_1...s_t)$
Bellman (MRP)	$V(s) = R(s) + \gamma\Sigma P(s'\|s)V(s')$
Bellman optimality	$V(s) = max_a[R(s,a) + \gamma\Sigma P(s'\|s,a)V(s')]$
Q-function	$Q^\pi(s,a) = E[G_t \mid s, a]$
Q-learning update	$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max Q(s',a') - Q(s,a)]$
Optimal policy	$\pi(s) = argmax_a\ Q(s,a)$

Memory Tricks

- RL loop = **A-E-S-A-R**; MDP = **SAP-R-γ**; MC → MRP → MDP = "+Reward, +Action".
- Learning types: **S**tudent-teacher, **U**nguided explorer, **R**eward-driven pet.
- Q update: "New = Old + α(Target − Old)".
- α = learning rate (how fast), γ = discount (how far-sighted), ε = exploration (how curious).

5. Assumptions & Gaps

- **Unit mapping:** Q7/Q8 of each paper are taken as Unit 4 (= syllabus Unit VI, Reinforcement Learning). The supervised/unsupervised comparison (U4-2) appears in this slot in all three papers, so it is kept in Unit 4.
- **2023 Q8(b)** ("Explain: supervised, unsupervised, reinforcement learning") and **2024 Q7(a)** (supervised vs unsupervised only) are merged into the three-way comparison U4-2.
- **2023 Q7(a)** and **2024 Q8(a)** (term lists) are merged into U4-5, which covers all six terms asked.
- All Unit 4 topics were found in ML Unit 3-6.pdf (Unit VI); no ⚠ flags.